GRID CLASSIFICATION

SVC

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
dataset=pd.read_csv("Social_Network_Ads.csv")
#dataset
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[1]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False
4	15804002	19	76000	0	True
...	...	...	...	...	...
395	15691863	46	41000	1	False
396	15706071	51	23000	1	True
397	15654296	50	20000	1	False
398	15755018	36	33000	0	True
399	15594041	49	36000	1	False

400 rows × 5 columns

In [2]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[2]:
```
Purchased
0    257
1    143
```

In [3]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[3]: (400, 4)

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

In [6]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

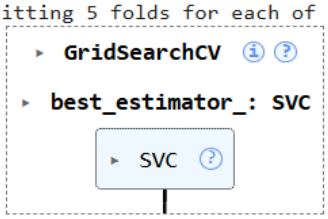
In [7]:
```
from sklearn.svm import SVC
```

In [8]:
```
param_grid = {'kernel':['rbf','poly','sigmoid','linear'],
              'C':[10,100,1000,2000],
              'gamma':['auto','scale']}
```

In [9]:
```
grid = GridSearchCV(SVC(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 32 candidates, totalling 160 fits
```

Out[9]:



In [10]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}:
```

In [11]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[11]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
       0, 1])
```

In [12]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)

from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[77  8]
 [ 4 45]]
              precision    recall  f1-score   support

           0       0.95      0.91      0.93        85
           1       0.85      0.92      0.88        49

    accuracy                           0.91       134
   macro avg       0.90      0.91      0.91       134
weighted avg       0.91      0.91      0.91       134
```

Decision tree

With Standard Scalar

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
```

In [3]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[3]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [4]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[4]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False
4	15804002	19	76000	0	True
...	...	...	...	...	...
395	15691863	46	41000	1	False
396	15706071	51	23000	1	True
397	15654296	50	20000	1	False
398	15755018	36	33000	0	True
399	15594041	49	36000	1	False

400 rows × 5 columns

In [5]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[5]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [6]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[6]:
```
(400, 4)
```

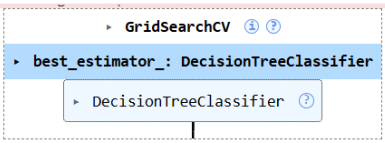
In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

In [16]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [17]:
```
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 3, 5, 7, 10],
    'min_samples_split':[2,3,4,5],
    'min_samples_leaf':[1, 1.5,2,2.5],
    'ccp_alpha': [0.0, 0.01, 0.02]  # Cost-complexity pruning parameter
}
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 480 candidates, totalling 2400 fits
```

Out[17]:



In [18]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'ccp_alpha': 0.0, 'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}:
```

In [19]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[19]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       0, 1])
```

In [20]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[78  7]
 [ 7 42]]
```

In [21]:
```
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.92      0.92      0.92        85
           1       0.86      0.86      0.86        49

    accuracy                           0.90       134
   macro avg       0.89      0.89      0.89       134
weighted avg       0.90      0.90      0.90       134
```

Without Standard Scalar

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
```

In [3]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[3]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [4]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[4]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False
4	15804002	19	76000	0	True
...	...	...	...	...	...
395	15691863	46	41000	1	False

In [5]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[5]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [6]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[6]: (400, 4)

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

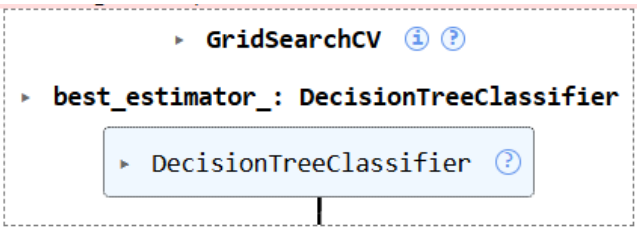
In [8]:
```
# from sklearn.preprocessing import StandardScaler
# sc = StandardScaler()
# x_train = sc.fit_transform(x_train)
# x_test = sc.transform(x_test)
```

In [11]:
```
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 3, 5, 7, 10],
    'min_samples_split':[2,3,4,5],
    'min_samples_leaf':[1, 1.5,2,2.5],
    'ccp_alpha': [0.0, 0.01, 0.02]  # Cost-complexity pruning parameter
}
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 480 candidates, totalling 2400 fits
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:489: FitFailedWarni
1200 fits failed out of a total of 2400.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
600 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 851,
    estimator.fit(X_train, y_train, **fit_params)
    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\base.py", line 1393, in wrapper
    estimator._validate_params()
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\base.py", line 554, in _validate_params
```

Out[11]:



In [12]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'ccp_alpha': 0.0, 'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}:
```

In [13]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[13]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       0, 1])
```

In [14]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[78  7]
 [ 7 42]]
```

In [15]:
```
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
              precision    recall  f1-score   support

           0       0.92      0.92      0.92        85
           1       0.86      0.86      0.86        49

    accuracy                           0.90       134
   macro avg       0.89      0.89      0.89       134
weighted avg       0.90      0.90      0.90       134
```

Random Forest

With Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False
4	15804002	19	76000	0	True
...	...	...	...	...	...
395	15691863	46	41000	1	False

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]:
```
(400, 4)
```

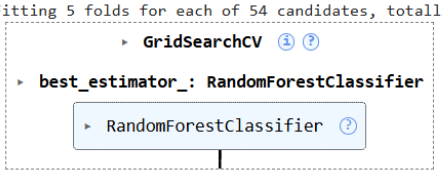
In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

In [13]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [14]:
```
from sklearn.ensemble import RandomForestClassifier
param_grid = {
    'n_estimators':[100, 150, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split':[2,3,4],
    'max_features': ['sqrt', 'log2']
}
grid = GridSearchCV(RandomForestClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 54 candidates, totalling 270 fits
```

Out[14]:



In [15]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'max_depth': None, 'max_features': 'sqrt', 'min_samples_split': 4, 'n_estimators': 200}:
```

In [16]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[16]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
       0, 1])
```

In [17]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[78  7]
 [ 5 44]]
              precision    recall  f1-score   support

           0       0.94      0.92      0.93        85
           1       0.86      0.90      0.88        49

    accuracy                           0.91       134
   macro avg       0.90      0.91      0.90       134
weighted avg       0.91      0.91      0.91       134
```

Without Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]: (400, 4)

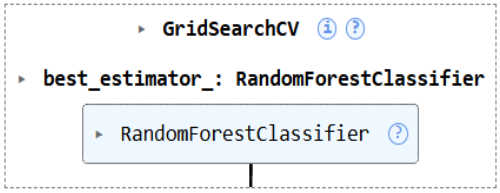
In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

In [7]:
```
# from sklearn.preprocessing import StandardScaler
# sc = StandardScaler()
# x_train = sc.fit_transform(x_train)
# x_test = sc.transform(x_test)
```

In [9]:
```
from sklearn.ensemble import RandomForestClassifier
param_grid = {
    'n_estimators':[100, 150, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split':[2,3,4],
    'max_features': ['sqrt', 'log2']
}
grid = GridSearchCV(RandomForestClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 54 candidates, totalling 270 fits
```

Out[9]:



In [10]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'max_depth': 20, 'max_features': 'sqrt', 'min_samples_split': 4, 'n_estimators': 200}:
```

In [11]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[11]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
       0, 1])
```

In [12]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[78  7]
 [ 5 44]]
              precision    recall  f1-score   support

           0       0.94      0.92      0.93        85
           1       0.86      0.90      0.88        49

    accuracy                           0.91       134
   macro avg       0.90      0.91      0.90       134
weighted avg       0.91      0.91      0.91       134
```

Logistic regression

With Standard Scalar

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [3]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[3]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [4]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[4]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False

In [5]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[5]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [6]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[6]:
```
(400, 4)
```

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

In [8]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

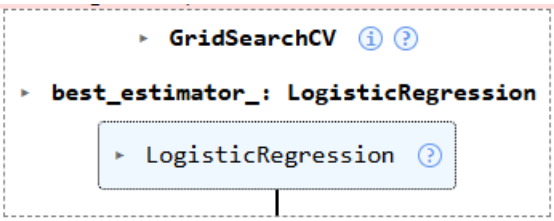
In [9]:
```
from sklearn.linear_model import LogisticRegression
param_grid = {
    'C': [1,1.5,2,2.5],
    'penalty': ['l1', 'l2', 'elasticnet', None],
    'solver': ['lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga']
}
```

In [10]:
```
grid = GridSearchCV(LogisticRegression(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 96 candidates, totalling 480 fits
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py:489: FitFailedWarning:
200 fits failed out of a total of 480.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
20 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_validation.py", line 851, in _fit_a
    estimator.fit(X_train, y_train, **fit_params)
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\base.py", line 1403, in wrapper
```

Out[10]:



In [11]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'C': 1, 'penalty': 'l1', 'solver': 'saga'}:
```

In [12]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[12]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1,
       0, 1])
```

In [13]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[79  6]
 [10 39]]
              precision    recall  f1-score   support

           0       0.89      0.93      0.91        85
           1       0.87      0.80      0.83        49

    accuracy                           0.88       134
   macro avg       0.88      0.86      0.87       134
weighted avg       0.88      0.88      0.88       134
```

Without Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]:
```
(400, 4)
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

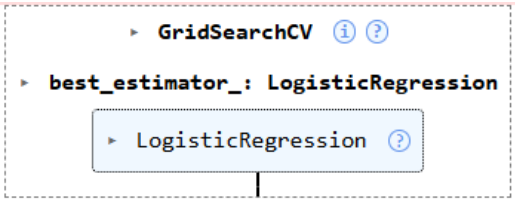
In [7]:
```
# from sklearn.preprocessing import StandardScaler
# sc = StandardScaler()
# X_train = sc.fit_transform(X_train)
# X_test = sc.transform(X_test)
```

In [11]:
```
from sklearn.linear_model import LogisticRegression
param_grid = {
    'C': [1,1.5,2,2.5],
    'penalty': ['l1', 'l2', 'elasticnet', None],
    'solver': ['lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'sag', 'saga']
}
```

In [12]:
```
grid = GridSearchCV(LogisticRegression(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 96 candidates, totalling 480 fits
```

Out[12]:



In [13]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'C': 2, 'penalty': 'l2', 'solver': 'lbfgs'}:
```

In [14]:
```
y_pred = grid.predict(X_test)
y_pred
```

Out[14]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1,
       0, 1])
```

In [15]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[79  6]
 [ 9 40]]
              precision    recall  f1-score   support

           0       0.90      0.93      0.91        85
           1       0.87      0.82      0.84        49

    accuracy                           0.89       134
   macro avg       0.88      0.87      0.88       134
weighted avg       0.89      0.89      0.89       134
```

KNN

With Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False
4	15804002	19	76000	0	True
...	...	...	...	...	...
395	15691863	46	41000	1	False
396	15706071	51	23000	1	True
397	15654296	50	20000	1	False
398	15755018	36	33000	0	True
399	15594041	49	36000	1	False

400 rows × 5 columns

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]:
```
(400, 4)
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

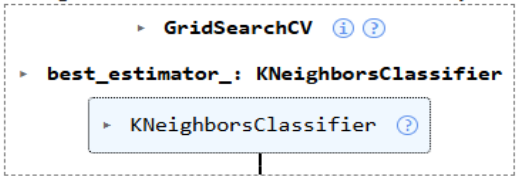
In [31]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [32]:
```
from sklearn.neighbors import KNeighborsClassifier
param_grid = {
    'n_neighbors': [2,7,11,17,21],
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute']
}
```

In [33]:
```
grid = GridSearchCV(KNeighborsClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
```

Out[33]:



In [34]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'algorithm': 'auto', 'n_neighbors': 7}:
```

In [35]:
```
y_pred = grid.predict(X_test)
```

In [36]:
```
y_pred
```

Out[36]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
       0, 1])
```

In [37]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[78  7]
 [ 4 45]]
              precision    recall  f1-score   support

           0       0.95      0.92      0.93        85
           1       0.87      0.92      0.89        49

    accuracy                           0.92       134
   macro avg       0.91      0.92      0.91       134
weighted avg       0.92      0.92      0.92       134
```

Without Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False
4	15804002	19	76000	0	True
...	...	...	...	...	...
395	15691863	46	41000	1	False
396	15706071	51	23000	1	True
397	15654296	50	20000	1	False
398	15755018	36	33000	0	True

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]:
```
(400, 4)
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

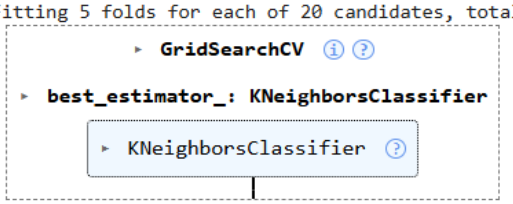
In [ ]:
```
# from sklearn.preprocessing import StandardScaler
# sc = StandardScaler()
# X_train = sc.fit_transform(X_train)
# X_test = sc.transform(X_test)
```

In [24]:
```
from sklearn.neighbors import KNeighborsClassifier
param_grid = {
    'n_neighbors': [2,7,11,17,21],
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute']
}
```

In [26]:
```
grid = GridSearchCV(KNeighborsClassifier(), param_grid, refit = True, verbose = 3,n_jobs=-1)
# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
```

Out[26]:



In [27]:
```
re=grid.cv_results_
print("The R_score value for best parameter {}:".format(grid.best_params_))
```

```
The R_score value for best parameter {'algorithm': 'auto', 'n_neighbors': 17}:
```

In [28]:
```
y_pred = grid.predict(X_test)
```

In [29]:
```
y_pred
```

Out[29]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1,
```

In [30]:
```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
from sklearn.metrics import classification_report
clf_report = classification_report(y_test, y_pred)
print(clf_report)
```

```
[[82  3]
 [15 34]]
              precision    recall  f1-score   support

           0       0.85      0.96      0.90        85
           1       0.92      0.69      0.79        49

    accuracy                           0.87       134
   macro avg       0.88      0.83      0.85       134
weighted avg       0.87      0.87      0.86       134
```

Navies Bayes

With Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True
1	15810944	35	20000	0	True
2	15668575	26	43000	0	False
3	15603246	27	57000	0	False

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]:
```
(400, 4)
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

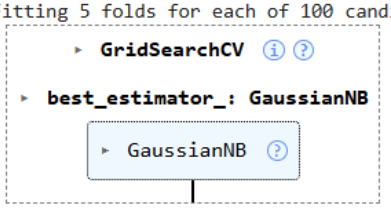
In [37]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
#sc = StandardScaler()
sc = MinMaxScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [38]:
```
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import ComplementNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.naive_bayes import CategoricalNB
```

In [39]:
```
#GaussisnNB
param_grid_gnb= {
    'var_smoothing': np.logspace(0, -9, num=100)
}
grid_gnb = GridSearchCV(GaussianNB(), param_grid_gnb, refit = True, verbose = 3,n_jobs=-1)
grid_gnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
```

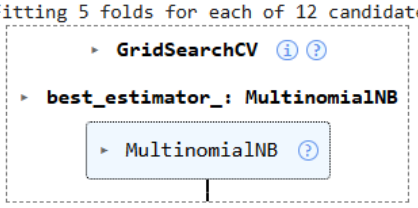
Out[39]:



In [40]:
```
#MultinomialNB
param_grid_mnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False]
}
grid_mnb = GridSearchCV(MultinomialNB(), param_grid_mnb, refit = True, verbose = 3,n_jobs=-1)
grid_mnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

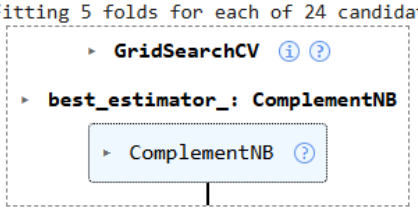
Out[40]:



In [41]:
```
#ComplementNB
param_grid_cnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False],
    'norm': [True, False]                 # Unique to ComplementNB
}
grid_cnb = GridSearchCV(ComplementNB(), param_grid_cnb, refit = True, verbose = 3,n_jobs=-1)
grid_cnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 24 candidates, totalling 120 fits
```

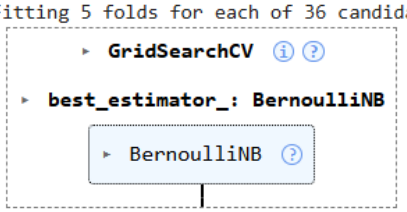
Out[41]:



In [42]:
```
#BernoulliNB
param_grid_bnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False],
    'binarize': [None, 0.0, 0.5]          # Threshold for converting features to binary
}
grid_bnb = GridSearchCV(BernoulliNB(), param_grid_bnb, refit = True, verbose = 3,n_jobs=-1)
grid_bnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits
```

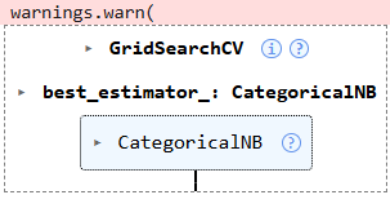
Out[42]:



In [43]:
```
#CategoricalNB
param_grid_catnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False]
}
grid_catnb = GridSearchCV(CategoricalNB(), param_grid_catnb, refit = True, verbose = 3,n_jobs=-1)
grid_catnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_search.py:1234: UserWarning: One or more of
  warnings.warn(
```

Out[43]:



In [44]:
```
re_gnb=grid_gnb.cv_results_
print("The R_score value of GaussisnNB for best parameter {}:".format(grid_gnb.best_params_))

re_mnb=grid_mnb.cv_results_
print("The R_score value of MultinomialNB for best parameter {}:".format(grid_mnb.best_params_))

re_cnb=grid_cnb.cv_results_
print("The R_score value of ComplementNB for best parameter {}:".format(grid_cnb.best_params_))

re_bnb=grid_bnb.cv_results_
print("The R_score value of BernoulliNB for best parameter {}:".format(grid_bnb.best_params_))

re_catnb=grid_catnb.cv_results_
print("The R_score value of CategoricalNB for best parameter {}:".format(grid_catnb.best_params_))
```

```
The R_score value of GaussisnNB for best parameter {'var_smoothing': np.float64(0.006579332246575682)}:
The R_score value of MultinomialNB for best parameter {'alpha': np.float64(100.0), 'fit_prior': True}:
The R_score value of ComplementNB for best parameter {'alpha': np.float64(100.0), 'fit_prior': True, 'norm': False}:
The R_score value of BernoulliNB for best parameter {'alpha': np.float64(10.0), 'binarize': None, 'fit_prior': False}:
The R_score value of CategoricalNB for best parameter {'alpha': np.float64(100.0), 'fit_prior': True}:
```

In [45]:
```
y_pred_gnb = grid_gnb.predict(X_test)
y_pred_gnb
```

Out[45]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
       0, 1])
```

In [46]:
```
y_pred_mnb = grid_mnb.predict(X_test)
y_pred_mnb
```

Out[46]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0])
```

In [47]:
```
y_pred_cnb = grid_cnb.predict(X_test)
y_pred_cnb
```

Out[47]:
```
array([0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1,
       1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0,
       1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0,
       1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0,
       1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0,
       0, 0])
```

In [48]:
```
y_pred_bnb = grid_bnb.predict(X_test)
y_pred_bnb
```

Out[48]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1,
       1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1,
       0, 1])
```

In [49]:
```
y_pred_catnb = grid_catnb.predict(X_test)
y_pred_catnb
```

Out[49]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0])
```

In [51]:
```
#GaussianNB
from sklearn.metrics import confusion_matrix
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
print(cm_gnb)

from sklearn.metrics import classification_report
clf_report_gnb = classification_report(y_test, y_pred_gnb)
print(clf_report_gnb)
```

```
[[81  4]
 [ 9 40]]
              precision    recall  f1-score   support

           0       0.90      0.95      0.93        85
           1       0.91      0.82      0.86        49

    accuracy                           0.90       134
   macro avg       0.90      0.88      0.89       134
weighted avg       0.90      0.90      0.90       134
```

In [52]:
```
#MultinomialNB
from sklearn.metrics import confusion_matrix
cm_mnb = confusion_matrix(y_test, y_pred_mnb)
print(cm_mnb)

from sklearn.metrics import classification_report
clf_report_mnb = classification_report(y_test, y_pred_mnb)
print(clf_report_mnb)
```

```
[[85  0]
 [49  0]]
              precision    recall  f1-score   support

           0       0.63      1.00      0.78        85
           1       0.00      0.00      0.00        49

    accuracy                           0.63       134
   macro avg       0.32      0.50      0.39       134
weighted avg       0.40      0.63      0.49       134
```

In [53]:
```
#ComplementNB
cm_cnb = confusion_matrix(y_test, y_pred_cnb)
print(cm_cnb)

clf_report_cnb = classification_report(y_test, y_pred_cnb)
print(clf_report_cnb)
```

```
[[40 45]
 [24 25]]
              precision    recall  f1-score   support

           0       0.62      0.47      0.54        85
           1       0.36      0.51      0.42        49

    accuracy                           0.49       134
   macro avg       0.49      0.49      0.48       134
weighted avg       0.53      0.49      0.49       134
```

In [54]:
```
#BernoulliNB
cm_bnb = confusion_matrix(y_test, y_pred_bnb)
print(cm_bnb)

clf_report_bnb = classification_report(y_test, y_pred_bnb)
print(clf_report_bnb)
```

```
[[69 16]
 [11 38]]
              precision    recall  f1-score   support

           0       0.86      0.81      0.84        85
           1       0.70      0.78      0.74        49

    accuracy                           0.80       134
   macro avg       0.78      0.79      0.79       134
weighted avg       0.80      0.80      0.80       134
```

In [55]:
```
#CategoricalNB
cm_catnb = confusion_matrix(y_test, y_pred_catnb)
print(cm_catnb)

clf_report_catnb = classification_report(y_test, y_pred_catnb)
print(clf_report_catnb)
```

```
[[85  0]
 [49  0]]
              precision    recall  f1-score   support

           0       0.63      1.00      0.78        85
           1       0.00      0.00      0.00        49

    accuracy                           0.63       134
   macro avg       0.32      0.50      0.39       134
weighted avg       0.40      0.63      0.49       134
```

Without Standard Scalar

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
```

In [2]:
```
dataset=pd.read_csv("Social_Network_Ads.csv")
dataset
```

Out[2]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased	Gender_Male
0	15624510	19	19000	0	True

In [4]:
```
dataset=dataset.drop("User ID",axis=1)
dataset["Purchased"].value_counts()
```

Out[4]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [5]:
```
indep=dataset[["Age","EstimatedSalary","Gender_Male"]]
dep=dataset["Purchased"]
dataset.shape
```

Out[5]: (400, 4)

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(indep, dep, test_size = 1/3, random_state = 0)
```

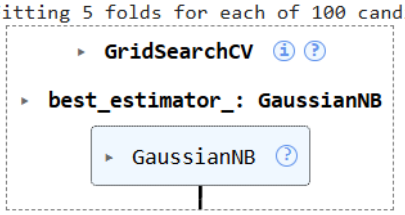
In [7]:
```
# from sklearn.preprocessing import StandardScaler
# sc = StandardScaler()
# X_train = sc.fit_transform(X_train)
# X_test = sc.transform(X_test)
```

In [8]:
```
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import ComplementNB
from sklearn.naive_bayes import BernoulliNB
from sklearn.naive_bayes import CategoricalNB
```

In [12]:
```
#GaussisnNB
param_grid_gnb= {
    'var_smoothing': np.logspace(0, -9, num=100)
}
grid_gnb = GridSearchCV(GaussianNB(), param_grid_gnb, refit = True, verbose = 3,n_jobs=-1)
grid_gnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
```

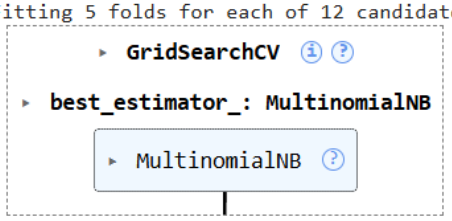
Out[12]:



In [13]:
```
#MultinomialNB
param_grid_mnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False]
}
grid_mnb = GridSearchCV(MultinomialNB(), param_grid_mnb, refit = True, verbose = 3,n_jobs=-1)
grid_mnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

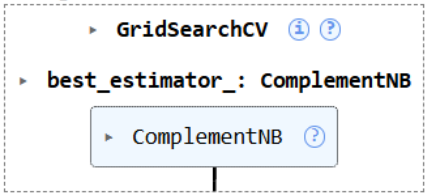
Out[13]:



In [14]:
```
#ComplementNB
param_grid_cnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False],
    'norm': [True, False]                 # Unique to ComplementNB
}
grid_cnb = GridSearchCV(ComplementNB(), param_grid_cnb, refit = True, verbose = 3,n_jobs=-1)
grid_cnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 24 candidates, totalling 120 fits
```

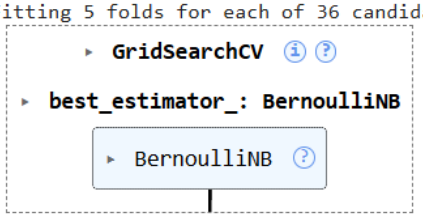
Out[14]:



In [15]:
```
#BernoulliNB
param_grid_bnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False],
    'binarize': [None, 0.0, 0.5]          # Threshold for converting features to binary
}
grid_bnb = GridSearchCV(BernoulliNB(), param_grid_bnb, refit = True, verbose = 3,n_jobs=-1)
grid_bnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 36 candidates, totalling 180 fits
```

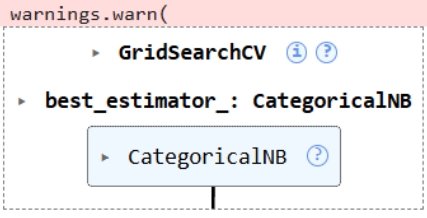
Out[15]:



In [16]:
```
#CategoricalNB
param_grid_catnb = {
    'alpha': np.logspace(2, -3, num=6),   # Generates [100.0, 10.0, 1.0, 0.1, 0.01, 0.001]
    'fit_prior': [True, False]
}
grid_catnb = GridSearchCV(CategoricalNB(), param_grid_catnb, refit = True, verbose = 3,n_jobs=-1)
grid_catnb.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\model_selection\_search.py:1234: UserWarning: One
  warnings.warn(
```

Out[16]:



In [17]:
```
re_gnb=grid_gnb.cv_results_
print("The R_score value of GaussisnNB for best parameter {}:".format(grid_gnb.best_params_))

re_mnb=grid_mnb.cv_results_
print("The R_score value of MultinomialNB for best parameter {}:".format(grid_mnb.best_params_))

re_cnb=grid_cnb.cv_results_
print("The R_score value of ComplementNB for best parameter {}:".format(grid_cnb.best_params_))

re_bnb=grid_bnb.cv_results_
print("The R_score value of BernoulliNB for best parameter {}:".format(grid_bnb.best_params_))

re_catnb=grid_catnb.cv_results_
print("The R_score value of CategoricalNB for best parameter {}:".format(grid_catnb.best_params_))
```

```
The R_score value of GaussisnNB for best parameter {'var_smoothing': np.float64(1.519911082952933e-08)}:
The R_score value of MultinomialNB for best parameter {'alpha': np.float64(1.0), 'fit_prior': True}:
The R_score value of ComplementNB for best parameter {'alpha': np.float64(100.0), 'fit_prior': True, 'norm': True}:
The R_score value of BernoulliNB for best parameter {'alpha': np.float64(100.0), 'binarize': None, 'fit_prior': True}:
The R_score value of CategoricalNB for best parameter {'alpha': np.float64(100.0), 'fit_prior': True}:
```

In [19]:
```
y_pred_gnb = grid_gnb.predict(X_test)
y_pred_gnb
```

Out[19]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1,
       0, 1])
```

In [20]:
```
y_pred_mnb = grid_mnb.predict(X_test)
y_pred_mnb
```

Out[20]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0])
```

In [21]:
```
y_pred_cnb = grid_cnb.predict(X_test)
y_pred_cnb
```

Out[21]:
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
       1, 1])
```

In [22]:
```
y_pred_bnb = grid_bnb.predict(X_test)
y_pred_bnb
```

```
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\naive_bayes.py:1256: RuntimeWarning: inv
  neg_prob = np.log(1 - np.exp(self.feature_log_prob_))
```

Out[22]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0])
```

In [25]:
```
from sklearn.metrics import confusion_matrix
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
print(cm_gnb)

from sklearn.metrics import classification_report
clf_report_gnb = classification_report(y_test, y_pred_gnb)
print(clf_report_gnb)
```

```
[[79  6]
 [11 38]]
              precision    recall  f1-score   support

           0       0.88      0.93      0.90        85
           1       0.86      0.78      0.82        49

    accuracy                           0.87       134
   macro avg       0.87      0.85      0.86       134
weighted avg       0.87      0.87      0.87       134
```

In [26]:
```
from sklearn.metrics import confusion_matrix
cm_mnb = confusion_matrix(y_test, y_pred_mnb)
print(cm_mnb)

from sklearn.metrics import classification_report
clf_report_mnb = classification_report(y_test, y_pred_mnb)
print(clf_report_mnb)
```

```
[[85  0]
 [49  0]]
              precision    recall  f1-score   support

           0       0.63      1.00      0.78        85
           1       0.00      0.00      0.00        49

    accuracy                           0.63       134
   macro avg       0.32      0.50      0.39       134
weighted avg       0.40      0.63      0.49       134
```

In [27]:
```
cm_cnb = confusion_matrix(y_test, y_pred_cnb)
print(cm_cnb)

clf_report_cnb = classification_report(y_test, y_pred_cnb)
print(clf_report_cnb)
```

```
[[ 4 81]
 [ 2 47]]
              precision    recall  f1-score   support

           0       0.67      0.05      0.09        85
           1       0.37      0.96      0.53        49

    accuracy                           0.38       134
   macro avg       0.52      0.50      0.31       134
weighted avg       0.56      0.38      0.25       134
```

In [28]:
```
cm_bnb = confusion_matrix(y_test, y_pred_bnb)
print(cm_bnb)

clf_report_bnb = classification_report(y_test, y_pred_bnb)
print(clf_report_bnb)
```

```
[[85  0]
 [49  0]]
              precision    recall  f1-score   support

           0       0.63      1.00      0.78        85
           1       0.00      0.00      0.00        49

    accuracy                           0.63       134
   macro avg       0.32      0.50      0.39       134
weighted avg       0.40      0.63      0.49       134
```

```
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\metrics\_classification
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\metrics\_classification
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0
C:\Users\Admin\anaconda3\Lib\site-packages\sklearn\metrics\_classification
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0
```

In [29]:
```
cm_catnb = confusion_matrix(y_test, y_pred_catnb)
print(cm_catnb)

clf_report_catnb = classification_report(y_test, y_pred_catnb)
print(clf_report_catnb)
```

```
[[85  0]
 [49  0]]
              precision    recall  f1-score   support

           0       0.63      1.00      0.78        85
           1       0.00      0.00      0.00        49

    accuracy                           0.63       134
   macro avg       0.32      0.50      0.39       134
weighted avg       0.40      0.63      0.49       134
```